- **Novel applications** (Sec. 6): We describe novel applications powered by Milvus. In particular, we build a series of **10** applications[2] on top of Milvus to demonstrate its broad applicability including image search, video search, chemical structure analysis, COVID-19 dataset search, personalized recommendation, biological multi-factor authentication, intelligent question answering, image-text retrieval, cross-modal pedestrian search, and recipe-food search.

2 SYSTEM DESIGN

In this section, we present an overview of Milvus. Figure 1 shows the architecture of Milvus with three major components: query engine, GPU engine, and storage engine. The query engine supports efficient query processing over vector data and it is optimized for modern CPUs by reducing cache misses and leveraging SIMD instructions. The GPU engine is a co-processing engine that accelerates performance with vast parallelism. It also supports multiple GPU devices for efficiency. The storage engine enables data durability and incorporates an LSM-based structure for dynamic data management. It runs on various file systems (including local file systems, Amazon S3, and HDFS) with a bufferpool in memory.

2.1 Query Processing

We first present the concept of entity used in Milvus and then explain query types, similarity functions, and application interfaces.

Entity. To best capture versatile data science and AI applications, Milvus supports query processing over both vector data and non-vector data. We define the term *entity* as follows to incorporate the two. Each entity in Milvus is described as one or more vectors and optionally some numerical attributes (non-vector data). For example, in the image search application, the numerical attributes can represent the age and height of a person in addition to possibly multiple machine-learned feature vectors of his/her photos (e.g., describing front-face, side-face, or posture [10]). In the current version of Milvus, we only support numerical attributes as observed from many applications. But in the future, we plan to support categorical attributes with indexes like inverted lists or bitmaps [64].

Query types. Milvus supports three primitive query types:

- Vector query: This query type is the traditional vector similarity search [33, 41, 48, 49], where each entity is described as a single vector. The system returns k most similar vectors where k is a user-input parameter.
- Attribute filtering: Each entity is specified by a single vector and some attributes [65]. The system returns k most similar vectors while adhering to the attributes constraints. As an example in recommender systems, users want to find similar clothes to a given query image while the price is below $100.
- Multi-vector query: Each entity is stored as multiple vectors [10]. The query returns top-k similar entities according to an aggregation function (e.g., weighted sum) between multiple vectors.

Similarity functions. Milvus offers commonly used similarity metrics, including Euclidean distance, inner product, cosine similarity, Hamming distance, and Jaccard distance, allowing applications to explore vector similarity in the most effective approach.

[2]https://github.com/milvus-io/bootcamp/tree/master/EN_solutions

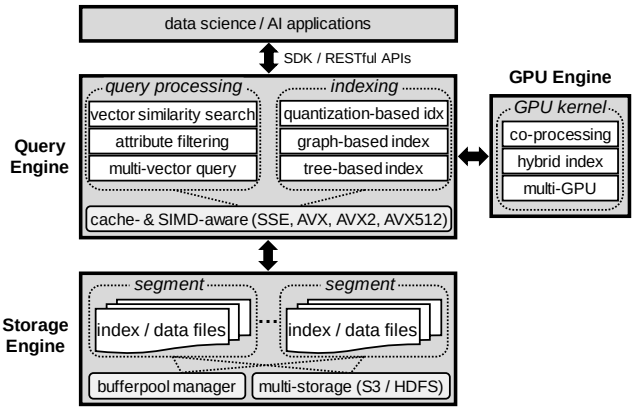

Figure 1: System architecture of Milvus

Application interfaces. Milvus provides easy-to-use SDK (software development kit) interfaces that can be directly called in applications written in various languages including Python, Java, Go, and C++. Milvus also supports RESTful APIs for web applications.

2.2 Indexing

Indexing is of tremendous importance to query processing in Milvus. But a challenging issue we face is to decide which indexes to support in Milvus, because there are numerous indexes developed for vector similarity search. The latest benchmark [41] shows that there is no winner in all scenarios and each index comes with tradeoffs in performance, accuracy, and space overhead.

In Milvus, we mainly support two types of indexes:[3] quantization-based indexes (including IVF_FLAT [3, 33, 35], IVF_SQ8 [3, 35], and IVF_PQ [3, 22, 33, 35]) and graph-based indexes (including HNSW [49] and RNSG [20]) to serve different applications. The design decision is based on factors including the latest literature review [41], industrial-strength systems (e.g., Alibaba PASE [68], Alibaba AnalyticDB-V [65], Jingdong Vearch [39]), open-source libraries (e.g., Facebook Faiss [3, 35]), and inputs from customers. We exclude LSH-based approaches because they have lower accuracy than quantization-based approaches on billion-scale data [65, 68].

Considering there are many new indexes coming out every year, Milvus is designed to easily incorporate the new indexes with a high-level abstraction. Developers only need to implement a few pre-defined interfaces for adding a new index. Our hope is that Milvus can eventually become a standard platform for vector data management with versatile indexes.

2.3 Dynamic Data Management

Milvus supports efficient insertions and deletions by adopting the idea of LSM-tree [47]. Newly inserted entities are stored in memory first as MemTable. Once the accumulated size reaches a threshold, or once every second, the MemTable becomes immutable and then gets flushed to disk as a new segment. Smaller segments are merged into larger ones for fast sequential access. Milvus implements a tiered merge policy (also used in Apache Lucene) that aims to merge segments of approximately equal sizes until a configurable size limit (e.g., 1GB) is reached. Deletions are supported in the same out-of-place approach except that the obsoleted vectors are

[3]Milvus also supports tree-based indexes, e.g., ANNOY [1].

removed during segment merge. Updates are supported by deletions and insertions. By default, Milvus builds indexes only for large segments (e.g., > 1GB) but users are allowed to manually build indexes for segments of any size if necessary. Both index and data are stored in the same segment. Thus, the segment is the basic unit of searching, scheduling, and buffering.

Milvus offers snapshot isolation to make sure reads and writes share a consistent view and do not interfere with each other. We present the details of snapshot isolation in Sec. 5.2.

2.4 Storage Management

As mentioned in Sec. 2.1, each entity is expressed as one or more vectors and optionally some attributes. Thus, each entity can be regarded as a row in an entity table. To facilitate query processing, Milvus physically stores the entity table in a columnar fashion.

Vector storage. For single-vector entities, Milvus stores all the vectors continuously without explicitly storing the row IDs. In this way, all the vectors are sorted by row IDs. Given a row ID, Milvus can directly access the corresponding vector since each vector is of the same length. For multi-vector entities, Milvus stores the vectors of different entities in a columnar fashion. For example, assuming that there are three entities (A, B, and C) in the database and each entity has two vectors $\mathbf{v}_1$ and $\mathbf{v}_2$, then all the $\mathbf{v}_1$ of different entities are stored together and all the $\mathbf{v}_2$ are stored together. That is, the storage format is $\{A.\mathbf{v}_1, B.\mathbf{v}_1, C.\mathbf{v}_1, A.\mathbf{v}_2, B.\mathbf{v}_2, C.\mathbf{v}_2\}$.

Attribute storage. The attributes are stored column by column. In particular, each attribute column is stored as an array of $\langle key, value \rangle$ pairs where the *key* is the attribute value and *value* is the row ID, sorted by the *key*. Besides that, we build skip pointers (i.e., min/max values) following Snowflake [16] as indexing for the data pages on disk. This allows efficient point query and range query in that column, e.g., price is less than $100.

Bufferpool. Milvus assumes that most (if not all) data and index are resident in memory for high performance. If not, it relies on an LRU-based buffer manager. In particular, the caching unit is a segment, which is the basic searching unit as explained in Sec. 2.3.

Multi-storage. For flexibility and reliability, Milvus supports multiple file systems including local file systems, Amazon S3, and HDFS for the underlying data storage. This also facilitates the deployment of Milvus in the cloud.

2.5 Heterogeneous Computing

Milvus is highly optimized for the heterogeneous computing platform that includes CPUs and GPUs. Sec. 3 presents the details.

2.6 Distributed System

Milvus can function as a distributed system deployed across multiple nodes. It adopts modern design practices in distributed systems and cloud systems such as storage/compute separation, shared storage, read/write separation, and single-writer-multi-reader. Sec. 5.3 explains more.

3 HETEROGENEOUS COMPUTING

In this section, we present the optimizations for Milvus to best leverage the heterogeneous computing platform involving both CPUs and GPUs to achieve high performance.

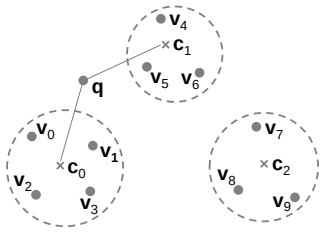

Figure 2: An example of quantization

As explained in Sec. 2.2, Milvus mainly supports quantization-based indexes (including IVF_FLAT [3, 33, 35], IVF_SQ8 [3, 35], and IVF_PQ [3, 22, 33, 35]) and graph-based indexes (including HNSW [49] and RNSG [20]). In this section, we use quantization-based indexes to illustrate our optimizations because they consume much less memory and are much faster to build index while achieving decent query performance when compared to graph-based indexes [65, 68]. Note that many optimizations (such as SIMD and GPU optimizations) can be applied to graph-based indexes.

3.1 Background

Before diving into optimizations, we explain vector quantization and quantization-based indexes. The main idea of vector quantization is to apply a quantizer z to map a vector $\mathbf{v}$ to a codeword $z(\mathbf{v})$ chosen from a codebook C [33]. The K-means clustering algorithm is commonly used to construct the codebook C where each codeword is the centroid and $z(\mathbf{v})$ is the closest centroid to $\mathbf{v}$. Figure 2 shows an example of 10 vectors ($\mathbf{v}_0$ to $\mathbf{v}_9$) of three clusters with centroids being $\mathbf{c}_0$ to $\mathbf{c}_2$, then $z(\mathbf{v}_0)$, $z(\mathbf{v}_1)$, $z(\mathbf{v}_2)$, or $z(\mathbf{v}_3)$ is $\mathbf{c}_0$.

Quantization-based indexes (such as IVF_FLAT [3, 33, 35], IVF_SQ8 [3, 35], and IVF_PQ [3, 22, 33, 35]) use two quantizers: coarse quantizer and fine quantizer. The coarse quantizer applies the K-means algorithm (e.g., K is 16384 in Milvus and Faiss [3]) to cluster vectors into K buckets. And the fine quantizer encodes the vectors within each bucket. Different indexes may use different fine quantizers. IVF_FLAT uses the original vector representation; IVF_SQ8 uses a compressed representation for the vectors by adopting one-dimensional quantizer (called "scalar quantizer") to compress a 4-byte float value to a 1-byte integer; and IVF_PQ uses product quantization that splits each vector into multiple sub-vectors and applies K-means for each sub-space.

Query processing (of a query $\mathbf{q}$) over quantization-based indexes takes two steps: (1) Find the closest n_{probe} buckets (or clusters) based on the distance between $\mathbf{q}$ and the centroid of each bucket. For example, assuming n_{probe} is 2 in Figure 2, then the closest two buckets of $\mathbf{q}$ are centered at $\mathbf{c}_0$ and $\mathbf{c}_1$. The parameter n_{probe} controls the tradeoff between accuracy and performance. Higher n_{probe} produces better accuracy but worse performance. (2) Search within each of the n_{probe} relevant buckets based on different fine quantizers. For example, if the index in Figure 2 is IVF_FLAT, then it needs to scan the vectors $\mathbf{v}_0$ to $\mathbf{v}_6$ in the two buckets.

3.2 CPU-oriented Optimizations

3.2.1 Cache-aware Optimizations in Milvus

The fundamental problem for query processing over quantization-based indexes is that, given a collection of m queries $\{\mathbf{q}_1, \mathbf{q}_2, ..., \mathbf{q}_m\}$ and a collection of n data vectors $\{\mathbf{v}_1, \mathbf{v}_2, ..., \mathbf{v}_n\}$, how to quickly

find for each query $\mathbf{q}_i$ its top-k similar vectors? In practice, users can submit batch queries so that $m \geq 1$.

This operation happens in finding the relevant buckets as well as searching within each relevant bucket. The original implementation in Facebook Faiss [3], which Milvus is built on top of, is inefficient because it incurs many CPU cache misses as explained below. Thus, Milvus develops an optimized approach to significantly reduce data movement between main memory and CPU caches.

Original implementation in Facebook Faiss [3]. Faiss uses the OpenMP multi-threading to process queries in parallel. Each thread is assigned to work on a single query at a time. The thread is released (for next query) once the current task is finished. Each task compares $\mathbf{q}_i$ with all the n data vectors and maintains a k-sized heap to store the results.

The above solution in Faiss has two performance issues: (1) It incurs many CPU cache misses, because for each query the entire data needs to be streamed through CPU caches and cannot be reused for the next query. Thus, each thread accesses m/t times of the entire data where t is the total number of threads. (2) It cannot fully leverage multi-core parallelism when the batch size m is small.

Optimizations in Milvus. Milvus develops two ideas to tackle the issues. First, it reuses the accessed data vectors as much as possible for multiple queries to minimize CPU cache misses. Specifically, it optimizes for reducing L3 cache misses because the penalty to access memory is high and also L3 cache size (typically 10s MB) is much bigger than L1/L2 cache, leaving more room for optimizations. Second, it uses fine-grained parallelism that assigns threads to data vectors instead of query vectors to best leverage multi-core parallelism, because the data size n is usually much bigger than the query size m in practice.

Figure 3 shows the overall design. Specifically, let t be the number of threads, then each thread T_i is assigned $b = n/t$ data vectors:[4] $\{\mathbf{v}_{(i-1)*b}, \mathbf{v}_{(i-1)*b+1}, ..., \mathbf{v}_{i*b-1}\}$. Milvus then partitions the m queries into query blocks of size s such that each query block (together with its associated heaps) can always fit in the L3 CPU cache. We decide s later on in Equation (1). Here we assume that m is divisible by s. Milvus computes the top-k results of each query block at a time with multiple threads. Whenever each thread loads its assigned data vectors to L3 cache, they will be compared against the entire query block (with s queries) in the cache. To minimize the synchronization overhead, Milvus assigns a heap per query per thread. In particular, assuming the i-th query block $\{\mathbf{q}_{(i-1)*s}, \mathbf{q}_{(i-1)*s+1}, ..., \mathbf{q}_{i*s-1}\}$ is in cache, Milvus dedicates the heap $H_{r-1,j-1}$ for the j-th query $\mathbf{q}_{(i-1)*s+j-1}$ on the r-th thread T_{r-1}. Thus, the results of a query $\mathbf{q}_i$ are spread over t threads of heaps. Thus, it needs to merge the heaps of each thread to obtain the final top-k results.

Next, we discuss how to determine the query block size s such that s queries and their associated heaps can always fit in L3 cache. Let d be the dimensionality, then the size of each query is $d \times$ sizeof(float). Since each heap entry contains a pair of vector ID and similarity, then the total size of the heaps (per query) is $t \times k \times$ (sizeof(int64) + sizeof(float)) where t is the number of threads. Thus, s is computed as follows:

$$s = \frac{\text{L3's cache size}}{d \times \text{sizeof(float)} + t \times k \times (\text{sizeof(int64)} + \text{sizeof(float)})}. \quad (1)$$

[4]We assume that n is divisible by t.

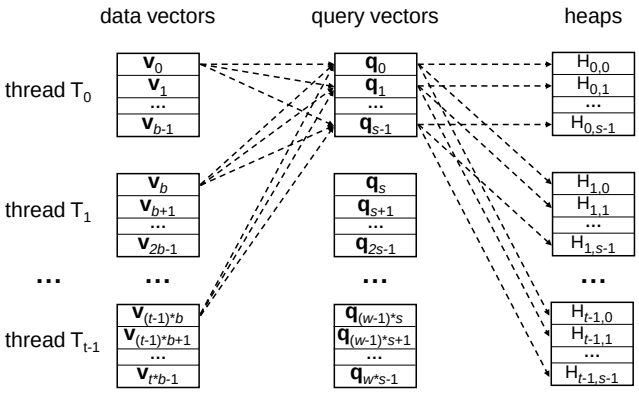

Figure 3: Cache-aware design in Milvus

In this way, each thread only accesses $m/(s*t)$ times of the entire data, which is s times smaller than the original implementation in Facebook Faiss [3]. Experiments (Sec. 7.4) show that this improves performance by a factor of 1.5× to 2.7×.

3.2.2 SIMD-aware Optimizations in Milvus

Modern CPUs support increasingly wider SIMD instructions. Thus, it is not surprising that Facebook Faiss [3] implements SIMD-aware algorithms to accelerate vector similarity search. We make two engineering optimizations in Milvus: (1) Supporting AVX512; and (2) Automatic SIMD-instruction selection.

Supporting AVX512. Faiss [3] does not support AVX512, which is now available in mainstream CPUs. Thus, we extend the similarity computing function with AVX512 instructions, such as `_mm512_add_ps`, `_mm512_mul_ps`, and `_mm512_extractf32x8_ps`. Now Milvus supports SIMD SSE, AVX, AVX2, and AVX512.

Automatic SIMD-instruction selection. Milvus is designed to work well on a wide spectrum of CPU processors (both on-premises and cloud platforms) with different SIMD instructions (e.g., SIMD SSE, AVX, AVX2, and AVX512). Thus the challenge is, given a single piece of software binary (i.e., Milvus), how to make it automatically invoke the suitable SIMD instructions on any CPU processor? Faiss [3] does not support it and users need to manually specify the SIMD flag (e.g., "-msse4") during compilation time. In Milvus, we take a considerable amount of engineering effort to refactor the codebase of Faiss. We factor out the common functions (e.g., similarity computing) that rely on SIMD accelerations. Then for each function, we implement four versions (i.e., SSE, AVX, AVX2, AVX512) and put each one into a separated source file, which is further compiled individually with the corresponding SIMD flag. During runtime, Milvus can automatically choose the suitable SIMD instructions based on the current CPU flags and then link the right function pointers using hooking.

3.3 GPU-oriented Optimizations

GPU is known for vast parallelism and Faiss [3] supports GPU for query processing over vector data. Milvus enhances Faiss in two aspects: (1) Supporting bigger k in the GPU kernel; (2) Supporting multi-GPU devices.

Supporting bigger k in GPU kernel. The original implementation in Faiss [3] does not support top-k query processing where k is greater than 1024 due to the limit of shared memory. But many

application such as video surveillance and recommender systems may need bigger k for further verification or re-ranking [69, 71].

Milvus overcomes this limitation and supports k up to 16384 although technically Milvus can support any k.[5] When k is larger than 1024, Milvus executes the query in multiple rounds to cumulatively produce the final results. In the first round, Milvus behaves the same as Faiss and gets the top 1024 results. For the second and later rounds, Milvus first checks the distance of the last result (denoted as d_l) in the previous round. Apparently, d_l is so far the largest distance in the partial results. To handle vectors with equivalent distance to the query, Milvus also records vector IDs in the result whose distances are equal to d_l. Then Milvus filters out vectors whose distances are smaller than d_l or IDs are recorded. From the remaining data, Milvus gets the next 1024 results. By doing so, Milvus ensures that results in previous rounds will not appear in the current round. After that, the new results are merged with the partial results obtained in earlier rounds. Milvus processes the query in a round-by-round fashion until a sufficient number of results are collected.

Supporting multi-GPU devices. Faiss [3] supports multiple GPU devices since they are usually found in modern servers. But Faiss needs to declare all the GPU devices in advance during compilation time. That means if the Faiss codebase is compiled using a server with c GPUs, then the software binary can only be running in a server that has at least c GPUs.

Milvus overcomes this limitation by allowing users to select any number of GPU devices during *runtime* (instead of compilation time). As a result, once the Milvus codebase is compiled into a software binary, it can run at any server. Under the hood, Milvus introduces a segment-based scheduling that assigns segment-based search tasks to the available GPU devices. Each segment can only be served by a single GPU device. This is particularly a good fit for the cloud environment with dynamic resource management where GPU devices can be elastically added or removed. For example, if there is a new GPU device installed, Milvus can immediately discover it and assign the next available search task to it.

3.4 GPU and CPU Co-design

In this mode, the GPU memory is not large enough to store the entire data. Facebook Faiss [3] alleviates the problem by using a low-footprint compressed index (called IVF_SQ8 [3])[6] and moving data from CPU memory to GPU memory (via PCIe bus) on demand. However, we find that there are two limitations: (1) The PCIe bandwidth is not fully utilized, e.g., our experiments show that the measured I/O bandwidth is only 1~2GB/s while PCIe 3.0 (16x) supports up to 15.75GB/s. (2) It is not always beneficial to execute queries on GPU (than CPU) considering the data transfer.

Milvus develops a new index called SQ8H (where 'H' stands for hybrid) to address the above limitations (Algorithm 1).

Addressing the first limitation. We investigate the codebase of Faiss and figure out that Faiss copies data (from CPU to GPU) bucket by bucket, which underutilizes the PCIe bandwidth since each bucket can be small. So the natural idea is to copy multiple buckets simultaneously. But the downside of such multi-bucket-copying is the handling of deletions where Faiss uses a simple in-place update approach because each bucket is copied (and stored) individually. Fortunately, deletions (and updates) are easily handled in Milvus since Milvus adopts an efficient LSM-based out-of-place approach (Sec. 2.3). As a result, Milvus improves the I/O utilization by copying multiple buckets if possible (line 3 of Algorithm 1).

Algorithm 1: SQ8H

```
1 let n_q be the batch size;
2 if n_q ≥ threshold then
3     run all the queries entirely in GPU (load multiple buckets
      to GPU memory on the fly);
4 else
5     execute the step 1 of SQ8 in GPU: finding n_probe buckets;
6     execute the step 2 of SQ8 in CPU: scanning every relevant
      bucket;
```

Addressing the second limitation. We observe that GPU outperforms CPU only if the query batch size is large enough considering the expensive data movement. That is because more queries make the workload more computation-intensive since they search the same data. Thus, if the batch size is bigger than a threshold (e.g., 1000), Milvus executes all the queries in GPU and loads necessary buckets if GPU memory is insufficient (line 2 of Algorithm 1). Otherwise, Milvus executes the query in a hybrid manner as follows. As mentioned in Sec. 3.1, there are two steps for searching quantization-based indexes: finding n_{probe} relevant (closest) buckets and scanning each relevant bucket. Milvus executes step 1 in GPU and step 2 in CPU because we observe that step 1 has a much higher computation-to-I/O ratio than step 2 (line 5 and 6 in Algorithm 1). That is because in step 1, all the queries compare against the *same* K centroids to find n_{probe} nearest buckets, and also the K centroids are small enough to be resident in the GPU memory. By contrast, data accesses in step 2 are more scattered since different queries do not necessarily access the same buckets.

4 ADVANCED QUERY PROCESSING

4.1 Attribute Filtering

As mentioned in Sec. 2.1, attribute filtering is a hybrid query type that involves both vector data and non-vector data [65]. It only searches vectors that satisfy the attributes constraints. It is crucial to many applications [65], e.g., finding similar houses (vector data) whose sizes are within a specific range (non-vector data). For presentation purpose, we assume that each entity is associated with a single vector and a single attribute since it is straightforward to extend the algorithms to multiple attributes. We defer multi-vector query processing to Sec. 4.2.

Formally, each such query involves two conditions C_A and C_V where C_A specifies the attribute constraint and C_V is the normal vector query constraint that returns top-k similar vectors. Without loss of generality, C_A is represented in the form of $a >= p_1$ && $a <= p_2$ where a is the attribute (e.g., size, price) and p_1 and p_2 are two boundaries of a range condition (e.g., $p_1 = 100$ and $p_2 = 500$).

There are several approaches to solve attribute filtering as recently studied in AnalyticDB-V [65]. In Milvus, we implement those

[5] In Milvus, we purposely limit k to 16384 to prevent large data movement over networks. Also, that number is sufficient for the applications we have seen so far.

[6] Note that IVF_SQ8 takes 1/4 the space of IVF_FLAT while losing only 1% recall. But the design principle and optimizations in Sec. 3.4 can be applicable to other quantization-based indexes such as IVF_FLAT and IVF_PQ.

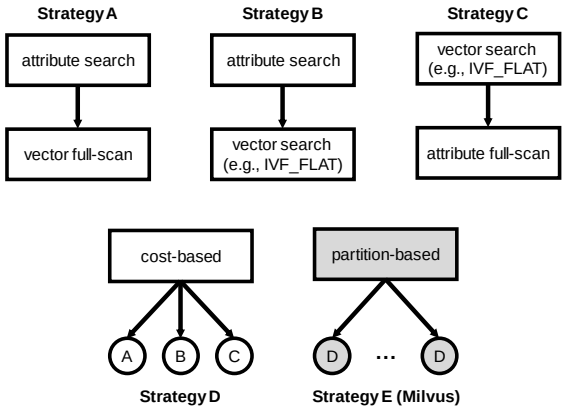

Figure 4: Different strategies for attribute filtering

approaches (i.e., strategies A, B, C, D as explained below). We then propose a partition-based approach (i.e., strategy E), which is up to 13.7× faster than the strategy D (i.e., state-of-the-art solution) according to the experiments in Sec. 7.5. Figure 4 shows a summary and we present the details next.

Strategy A: attribute-first-vector-full-scan. It only uses the attribute constraint C_A to obtain relevant entities via index search. Since the data is stored mostly in memory, we use binary search, but a B-tree index is also possible. When data cannot fit in memory, we use skip pointers for fast search. After that, all the entities in the result set are fully scanned to compare against the query vector to produce the final top-k results. Although simple, this approach is suitable when C_A is highly selective such that only a small number of candidates are required for further verification. Another interesting property of this strategy is that it produces the exact results.

Strategy B: attribute-first-vector-search. The difference with the strategy A is that after it obtains the relevant entities according to attribute constraint C_A, it produces a bitmap of the resultant entity IDs. Then it conducts the normal vector query processing based on C_V and checks the bitmap whenever a vector is encountered. Only vectors that pass bitmap testing are included in the final top-k results. This strategy is suitable in many cases when C_A or C_V is moderately selective.

Strategy C: vector-first-attribute-full-scan. In contrast to the strategy A, this approach only uses the vector constraint C_V to obtain the relevant entities via vector indexing like IVF_FLAT. Then the resultant entities are fully scanned to verify if they satisfy the attribute constraint C_A. To make sure there are k final results, it searches for $\theta \cdot k$ ($\theta > 1$) results during the vector query processing. This strategy is suitable when the vector constraint C_V is highly selective that the number of candidates is relatively small.

Strategy D: cost-based. It is a cost-based approach that estimates the cost of the strategy A, B, C, and picks up the one with the least cost as proposed in AnalyticDB-V [65]. From [65] and our experiments, the cost-based strategy is suitable in almost all cases.

Strategy E: partition-based. This is a partition-based approach that we develop in Milvus. The main idea is that it partitions the dataset based on the frequently searched attribute and applies the cost-based approach (i.e., the strategy D) for each partition. In particular, we maintain the frequency of each searched attribute in a hash table and increase the counter whenever a query refers to that attribute. Given a query of attribute filtering, it only searches the partitions whose attribute-ranges overlap with the query range. More importantly, if the range of a specific partition is covered by the query range, then this strategy does not need to check the attribute constraint (C_A) anymore and only focuses on vector query processing (C_V) in that partition, because all the vectors in that partition satisfy the attribute constraint.

As an example, suppose that there are many queries involving the attribute 'price' and the strategy E splits the dataset into five partitions: $\mathcal{P}_0$[1~100], $\mathcal{P}_1$[101~200], $\mathcal{P}_2$[201~300], $\mathcal{P}_3$[301~400], $\mathcal{P}_4$[401~500]. Then if the attribute constraint (C_A) of the query is [50~250], then only $\mathcal{P}_0$, $\mathcal{P}_1$, and $\mathcal{P}_2$ are necessary for searching because their ranges overlap with the query range. And when searching $\mathcal{P}_1$, there is no need to check the attribute constraint since its range is completely covered by the query range. This can significantly improve the query performance.

In the current version of Milvus, we create the partitions offline based on historical data and serve query processing online. The number of partitions (denoted as ρ) is a parameter configured by users. Choosing a proper ρ is subtle: If ρ is too small, then each partition contains too many vectors and it becomes hard to prune irrelevant partitions for this strategy; If ρ is too big, then the number of vectors in each partition is so small that the vector indexing deteriorates towards linear search. Based on our experience, we recommend ρ to be chosen such that each partition contains roughly 1 million vectors. For example, on a billion-scale dataset, there are around 1000 partitions. However, it is an interesting future work to investigate the use of machine learning and statistics to dynamically partition the data and decide the right number of partitions.

4.2 Multi-vector Queries

In many applications, each entity is specified by multiple vectors for accuracy. For example, intelligent video surveillance applications use different vectors to describe the front face, side face, and posture for each person captured by camera [10]. Recipe search applications use multiple vectors to represent text description and associated images for each recipe [56]. Another source of multi-vector is that many applications use more than one machine learning model even for the same object to best describe that object [30, 69].

Formally, each entity contains μ vectors $\mathbf{v}_0$, $\mathbf{v}_1$, ..., $\mathbf{v}_{\mu-1}$. Then a multi-vector query finds top-k entities according to an aggregated scoring function g over the similarity function f (e.g., inner product) of each individual vector $\mathbf{v}_i$. Specifically, the similarity of two entities X and Y is computed as $g(f(X.\mathbf{v}_0, Y.\mathbf{v}_0), ..., f(X.\mathbf{v}_{\mu-1}, Y.\mathbf{v}_{\mu-1}))$ where $X.\mathbf{v}_i$ means the vector $\mathbf{v}_i$ of the entity X. To capture a wide range of applications, we assume the aggregation function g to be monotonic in the sense that g is non-decreasing with respect to every $f(X.\mathbf{v}_i, Y.\mathbf{v}_i)$ [19]. In practice, many commonly used aggregation functions are monotonic, e.g., weighted sum, average/median, and min/max.

Naive solution. Let $\mathcal{D}$ be the dataset and $\mathcal{D}_i$ is a collection of $\mathbf{v}_i$ of all the entities, i.e., $\mathcal{D}_i = \{e.\mathbf{v}_i | e \in \mathcal{D}\}$. Given a query q, the naive solution is to issue an individual top-k query for each vector $q.\mathbf{v}_i$ on $\mathcal{D}_i$ to produce a set of candidates, which are further computed to obtain the final top-k results. Although simple, it can miss many true results leading to extremely low recall (e.g., 0.1). This approach

was widely used in the area of AI and machine learning to support effective recommendations, e.g., [29, 70].

In Milvus, we develop two new approaches, namely vector fusion and interactive merging that target for different scenarios.

Vector fusion. We illustrate the vector fusion approach assuming that the similarity function is inner product and we will explain how to extend to other similarity functions afterwards. Let e be an arbitrary entity in the dataset and $\mathbf{v}_0$, $\mathbf{v}_1$, ..., $\mathbf{v}_{\mu-1}$ be the μ vectors that each entity contains, this approach stores for each entity e its μ vectors as a concatenated vector $\mathbf{v} = [e.\mathbf{v}_0, e.\mathbf{v}_1, ..., e.\mathbf{v}_{\mu-1}]$. Let q be a query entity, during query processing, this approach applies the aggregation function g to the μ vectors of q, producing an aggregated query vector. For example, if the aggregation function is weighted sum with w_i for each weight, then the aggregated query vector is: $[w_0 \times q.\mathbf{v}_0, w_1 \times q.\mathbf{v}_1, ..., w_{\mu-1} \times q.\mathbf{v}_{\mu-1}]$. Then it searches the aggregated query vector against the concatenated vectors in the dataset to obtain the final results. It is straightforward to prove the correctness of vector fusion because the similarity function of inner product is decomposable.

The vector fusion approach is simple and efficient because it only needs to invoke the vector query processing once. But it requires a decomposable similarity function such as inner product. This sounds restrictive but when the underlying data is normalized, many similarity functions such as cosine similarity and Euclidean distance can be converted to inner product equivalently.

Iterative merging. If the underlying data is not normalized *and* the similarity function is not decomposable (e.g., Euclidean distance), then the above vector fusion approach is not applicable. Then we develop another algorithm called iterative merging (see Algorithm 2) that is built on top of Fagin's well-known NRA algorithm [19], a general technique for top-k query processing.[7]

Our initial try is actually to use the NRA algorithm [19] by treating the results of each $q.\mathbf{v}_i$ on $\mathcal{D}_i$ as a stream provided by Milvus. However, we quickly find that it is inefficient because NRA frequently calls `getNext()` to obtain the next result of $q.\mathbf{v}_i$ interactively. However, existing vector indexing techniques such as quantization-based indexes and graph-based indexes do not support `getNext()` efficiently. A full search is required to get the next result. Another drawback of NRA is that, it incurs significant overhead to maintain the heap since every access in NRA needs to update the scores of the current objects in the heap.

Thus, iterative merging makes two optimizations over NRA: (1) It does not rely on `getNext()` and instead calls VectorQuery($q.\mathbf{v}_i$, $\mathcal{D}_i$, k') with adaptive k' to get the top-k' query results of $q.\mathbf{v}_i$. As a result, it does not need to invoke the vector query processing for every access as NRA does. It can also eliminate the expensive overhead of the heap maintenance as in NRA. (2) It introduces an upper bound of the maximum number of steps to access since the query results in Milvus are approximate.

Algorithm 2 shows iterative merging. The main idea is that it iteratively issues a top-k' query processing for each $q.\mathbf{v}_i$ on $\mathcal{D}_i$ and puts the results to $\mathcal{R}_i$, where $\mathcal{D}_i$ is a collection of $\mathbf{v}_i$ of all the entities in the dataset $\mathcal{D}$, i.e., $\mathcal{D}_i = \{e.\mathbf{v}_i | e \in \mathcal{D}\}$, see line 3 and 4 in Algorithm 2. Then it executes the NRA algorithm over all the $\mathcal{R}_i$. If at least k results can be fully determined (line 5), i.e., NRA can safely stop, then the algorithm can terminate since top-k results can be produced. Otherwise, it doubles k' and iterates the process until k' reaches to a pre-defined threshold (line 2).

[7]Note that the TA algorithm in [19] cannot be applied in this setting because TA requires random access that is not available here.

Algorithm 2: Iterative merging

```
1 k' ← k;
2 while k' < threshold do
      // run top-k' processing for each q.v_i on D_i
3     foreach i do
4         R_i ← VectorQuery(q.v_i, D_i, k');
5     if k results are fully determined with NRA [19] on all R_i
      then
6         return top-k results;
7     else
8         k' ← k' × 2;
9 return top-k results from ∪_i R_i;
```

In contrast to the vector fusion approach, the iterative merging approach makes no assumption on the data and similarity functions, thus it can be used in a wide spectrum of scenarios. But the performance will be worse than vector fusion when the similarity function is decomposable.

Note that in the database field, there are many top-k algorithms proposed, e.g., [5, 12, 31, 42, 62]. However, those algorithms cannot be directly used to solve the multi-vector query processing, because the underlying vector indexes cannot support `getNext()` efficiently as mentioned earlier. The proposed iterative merging approach (Algorithm 2) is a generic framework so that it is possible to incorporate other top-k algorithms (e.g., [42]) by replacing line 5. But it remains an open question in terms of optimality and it is also interesting to optimize multi-vector query processing in the future.

5 SYSTEM IMPLEMENTATION

In this section, we present the implementation details of asynchronous processing, snapshot isolation, and distributed computing.

5.1 Asynchronous Processing

Milvus is designed to minimize the foreground processing via asynchronous processing to improve throughput. When Milvus receives heavy write requests, it first materializes the operations (similar to database logs) to disk and then acknowledges to users. There is a background thread that consumes the operations. As a result, users may not immediately see the inserted data. To prevent this, Milvus provides an API flush() that blocks all the incoming requests until the system finishes processing all the pending operations. Besides that, Milvus builds indexes asynchronously.

5.2 Snapshot Isolation

Milvus provides snapshot isolation to make sure reads and writes see a consistent view since Milvus supports dynamic data management. Every query only works on the snapshot when the query starts. Subsequent updates to the system will create new snapshots and do not interfere with the on-going queries.

Milvus manages dynamic data following the LSM-style. All the new data are inserted to memory first and then flushed to disk as immutable segments. Each segment has multiple versions and a new